DIGITLAB – Integrated BDML Mini Project Report

Project Title: Handwritten Digit Recognition Using Machine Learning and Deep Learning

Below is the complete report content based on your project code. You can copy it into Microsoft Word or Google Docs and format it for your college submission.

TITLE PAGE:

DIGITLAB

INTEGRATED BDML MINI PROJECT

HANDWRITTEN DIGIT RECOGNITION USING MACHINE LEARNING AND DEEP LEARNING

PROJECT REPORT

Submitted by

Name: MELDONA L

Register Number: 241501108

Department:AIML

ABSTRACT

DigitLab is an integrated mini project developed to demonstrate handwritten digit recognition using machine learning and deep learning techniques. The project uses the MNIST dataset, which contains grayscale images of handwritten digits from 0 to 9. The dataset is preprocessed and used to train and evaluate different classification algorithms.

The project implements Support Vector Machine (SVM), Random Forest, and Decision Tree classifiers and compares their performance using accuracy measurements and a confusion matrix. It also applies K-Means clustering and Principal Component Analysis (PCA) to explore patterns in the dataset. A feedforward neural network and a Convolutional Neural Network (CNN) are developed for digit classification. In addition, a Generative Adversarial Network (GAN) is implemented to generate digit-like images.

The project further includes SVM hyperparameter tuning using GridSearchCV and cross-validation. Finally, a Flask REST API is developed to provide digit predictions using the trained CNN model. The system saves trained models, experimental results, and deployment-support files. This project provides practical experience in data preprocessing, model training, evaluation, image generation, and API development.

1. INTRODUCTION

Handwritten digit recognition is an important application of machine learning and image processing. It involves identifying numerical digits from handwritten images and classifying them into the appropriate categories from 0 to 9.

Handwriting varies from person to person in shape, size, thickness, and writing style. Therefore, recognizing handwritten digits requires algorithms that can learn patterns and distinguish between different digit classes.

DigitLab is designed to explore different machine learning and deep learning approaches for handwritten digit recognition. The project uses Python and the MNIST dataset to train multiple models, compare their performance, visualize their predictions, and develop an API for accessing the trained CNN model.

The project combines nine experiments into a single workflow, covering data preprocessing, supervised learning, unsupervised learning, neural networks, generative modeling, hyperparameter optimization, and API development.

2. PROBLEM STATEMENT

To develop an integrated handwritten digit recognition system using the MNIST dataset and various machine learning and deep learning algorithms. The system aims to classify handwritten digits, compare model performance, explore clustering and image generation, optimize model parameters, and provide predictions through a Flask REST API.

3. AIM

To implement a handwritten digit recognition system using machine learning and deep learning techniques and demonstrate the complete workflow from data preprocessing to model deployment preparation.

4. OBJECTIVES

- To load and preprocess the MNIST handwritten digit dataset.
- To implement SVM, Random Forest, and Decision Tree classifiers.
- To compare classification models using accuracy and a confusion matrix.
- To apply K-Means clustering and PCA for data analysis and visualization.
- To develop a feedforward neural network and a CNN for digit classification.
- To implement a GAN for generating digit-like images.
- To perform SVM hyperparameter tuning using GridSearchCV.
- To evaluate models using test data and cross-validation.
- To save trained models and experimental results.
- To develop and test a Flask REST API for handwritten digit prediction.
- To generate configuration files for future Docker-based deployment.

5. SCOPE OF THE PROJECT

The scope of DigitLab includes the implementation and comparison of multiple machine learning and deep learning algorithms using the MNIST dataset.

The system performs digit classification, explores patterns through clustering and dimensionality reduction, generates synthetic digit-like images, and optimizes SVM parameters. It also demonstrates how a trained CNN model can be accessed through a REST API.

The current implementation is an educational prototype developed using Python in Google Colab. It creates a Dockerfile and the necessary application files, but building and deploying the Docker image are not performed by the notebook.

6. SOFTWARE AND HARDWARE REQUIREMENTS

6.1 Software Requirements

Software / Library	Purpose
Python	Main programming language
Google Colab	Development and execution environment
TensorFlow and Keras	Neural networks, CNN, GAN and model saving
Scikit-learn	Machine learning algorithms and evaluation
NumPy	Numerical computation and array manipulation
Matplotlib	Data visualization and plotting
Flask	REST API development
Joblib	Saving machine learning models
JSON	Storing experimental results
Docker	Containerization support

6.2 Hardware Requirements

- A computer or laptop.

- Internet connection for accessing Google Colab and downloading the dataset.
- Sufficient memory and processing resources for model training.
- A GPU is optional; Google Colab can provide accelerated hardware when available.

7. DATASET DESCRIPTION

The project uses the MNIST handwritten digit dataset, which is commonly used for evaluating image classification algorithms.

The dataset contains:

- 60,000 training images.
- 10,000 testing images.
- 10 digit classes, from 0 to 9.
- Grayscale images of size 28 × 28 pixels.

Each image contains 784 pixel values. In the original dataset, pixel intensities range from 0 to 255.

7.1 Data Preprocessing

The following preprocessing operations are performed:

1. The MNIST dataset is loaded using TensorFlow/Keras.
2. Pixel values are converted to the `float32` data type.
3. Pixel values are normalized by dividing them by 255.
4. Images are flattened into 784 features for classical machine learning and the feedforward neural network.
5. Images are retained in 28 × 28 × 1 format for the CNN.
6. Smaller subsets of the training and test data are used for selected experiments to reduce computational requirements.

The classical models use 5,000 training images and 2,000 test images. The feedforward neural network uses 12,000 training images, while the CNN uses 15,000 training images.

8. SYSTEM METHODOLOGY

The project follows a sequence of nine experiments.

MNIST Dataset

Data Preprocessing

Normalization and reshaping

Machine Learning Models

SVM, Random Forest and Decision Tree

Deep Learning Models

Feedforward NN and CNN

Additional Experiments

K-Means, PCA, GAN and SVM tuning

Flask REST API

Digit prediction and response validation

9. IMPLEMENTATION OF EXPERIMENTS

9.1 Experiment 1: Data Preprocessing

Aim: To load and preprocess the MNIST dataset for handwritten digit recognition.

Description:

The MNIST dataset is loaded using `tensorflow.keras.datasets.mnist`. The training and testing images are normalized by dividing their pixel values by 255. The images are converted into floating-point arrays.

For classical machine learning algorithms, each 28 × 28 image is flattened into a one-dimensional array containing 784 features. A set of sample images is displayed using Matplotlib to visualize the handwritten digits.

Outcome: The dataset is prepared for subsequent machine learning and deep learning experiments.

9.2 Experiment 2: SVM and Random Forest Classification

Aim: To implement and compare SVM and Random Forest classifiers.

Description:

The Support Vector Machine classifier uses an RBF kernel with `C=3` and `gamma='scale'`. It is trained on 5,000 training images.

The Random Forest classifier uses 100 decision trees and a fixed random seed for reproducibility. Both models predict digit classes for the first 2,000 test images.

The accuracy of each model is calculated using Scikit-learn's `accuracy_score` function. The trained Random Forest model is saved using Joblib.

Outcome: The classification accuracy of the SVM and Random Forest models is calculated and made available for comparison.

9.3 Experiment 3: Decision Tree Classification

Aim: To implement a Decision Tree classifier for handwritten digit recognition.

Description:

A Decision Tree classifier is created with a maximum depth of 20. It is trained using the same 5,000-image subset used for the classical models.

The trained model predicts the first 2,000 test images. The predicted labels are compared with the actual labels to calculate the classification accuracy.

Outcome: The Decision Tree model is evaluated, and its accuracy is recorded for comparison with other classifiers.

9.4 Experiment 4: Model Comparison and Confusion Matrix

Aim: To compare the classical classification models and visualize their performance.

Description:

The accuracy values of SVM, Random Forest, and Decision Tree are displayed in a bar chart using Matplotlib.

A confusion matrix is generated for the Random Forest classifier. It represents the relationship between the actual digit labels and the predicted digit labels, helping identify which digits are correctly classified and which are confused with other digits.

Outcome: The performance of the classical models is visually compared using a bar chart and confusion matrix.

9.5 Experiment 5: K-Means Clustering and PCA

Aim: To explore digit-image patterns using clustering and dimensionality reduction.

Description:

K-Means clustering is applied to the first 2,000 training images with ten clusters. It groups images according to similarities in their feature representations without using the actual digit labels.

Principal Component Analysis reduces the 784-dimensional image data to two principal components. The resulting two-dimensional representation is plotted using Matplotlib, with colors representing the K-Means cluster assignments.

The explained variance ratio of the two principal components is also calculated.

Outcome: The experiment visualizes groups of similar images in a two-dimensional space. The cluster numbers represent unsupervised groups and do not necessarily match the actual digit labels.

9.6 Experiment 6A: Feedforward Neural Network

Aim: To develop a feedforward neural network for digit classification.

Description:

A Sequential neural network is implemented using Keras. It contains an input layer with 784 features, two hidden Dense layers with 128 and 64 neurons, and an output layer with ten neurons.

ReLU activation is used in the hidden layers, while softmax activation is used in the output layer to produce class probabilities.

The model is compiled using the Adam optimizer and sparse categorical cross-entropy loss. It is trained for three epochs using 12,000 training images, with 10% of the training subset used for validation.

The model is evaluated using the complete test dataset.

Outcome: The trained feedforward neural network produces predictions for the ten digit classes, and its test accuracy is calculated.

9.7 Experiment 6B: Convolutional Neural Network (CNN)

Aim: To implement a CNN for handwritten digit recognition.

Description:

A Convolutional Neural Network is developed to learn spatial features from handwritten digit images.

The architecture consists of:

1. An input layer accepting 28 × 28 × 1 images.
2. A convolutional layer with 32 filters and ReLU activation.
3. A max-pooling layer.
4. A convolutional layer with 64 filters and ReLU activation.
5. Another max-pooling layer.
6. A flattening layer.
7. A Dense layer with 128 neurons.
8. A dropout layer with a rate of 0.25.
9. An output layer with ten neurons and softmax activation.

The CNN uses the Adam optimizer and sparse categorical cross-entropy loss. It is trained for three epochs on 15,000 training images, with a 10% validation split.

After training, the model is evaluated on the full MNIST test dataset. Training and validation accuracy are plotted, and predictions for ten test images are displayed alongside their actual labels.

The trained CNN is saved as `cnn_model.keras`.

Outcome: A CNN model is trained for handwritten digit classification, evaluated on the test dataset, and saved for API integration.

9.8 Experiment 7: Generative Adversarial Network (GAN)

Aim: To implement a GAN that generates digit-like images.

Description:

A Generative Adversarial Network consists of two neural networks: a generator and a discriminator.

The generator receives random noise of dimension 100 and transforms it into a 784-value output representing a flattened image. It uses Dense layers and a tanh output activation.

The discriminator receives flattened images and attempts to distinguish real MNIST images from generated images. It uses Dense layers, LeakyReLU activation, and a sigmoid output.

The GAN is trained using binary cross-entropy loss and Adam optimizers. The experiment uses 3,000 MNIST images and runs for two epochs.

After training, ten digit-like images are generated and displayed. The generator model is saved as `digit_generator.keras`.

Outcome: The experiment demonstrates adversarial training and synthetic digit-image generation.

9.9 Experiment 8: SVM Hyperparameter Tuning and Cross-Validation

Aim: To identify suitable SVM hyperparameters using GridSearchCV.

Description:

GridSearchCV is used to evaluate different combinations of SVM parameters. The parameter grid includes:

- `C`: 1 and 3.
- `kernel`: linear and RBF.

Three-fold StratifiedKFold cross-validation is applied with shuffling enabled. Accuracy is used as the scoring metric.

The best parameter combination and cross-validation accuracy are displayed. The selected SVM model is then evaluated on the first 2,000 test images and saved as `best_svm.joblib`.

Outcome: The experiment identifies the best-performing parameter combination among those tested and evaluates the selected model on the test subset.

9.10 Experiment 9: Flask REST API and Docker Configuration

Aim: To create and test a REST API for handwritten digit prediction.

Description:

A Flask application is generated in `app.py`. It loads the previously saved CNN model and exposes three endpoints.

Endpoint	Method	Function
`/`	GET	Returns project information and available endpoints.
`/health`	GET	Reports the API health status and model loading status.
`/predict`	POST	Predicts a digit from an input image array.

The prediction endpoint accepts JSON data containing an `image` field with exactly 784 numeric values. It validates the input, reshapes the image to 28 × 28 × 1, and normalizes values in the 0–255 range when necessary.

The response contains the predicted digit, its highest softmax probability as the confidence score, and the probabilities for all ten classes.

The notebook tests the endpoints using Flask's test client, including an invalid input case. It also generates `requirements.txt` and a Dockerfile to support future containerization.

Outcome: The Flask API source code and Docker configuration are generated, and endpoint behavior is tested without starting a separate web server. Docker image building and deployment are not performed.

10. RESULTS AND DISCUSSION

The project evaluates several classification models and produces visualizations to analyze their performance. It also demonstrates clustering, dimensionality reduction, generative image creation, and API-based prediction.

The classification accuracies are calculated during execution. Fill in the following table using the actual values printed by your Google Colab notebook.

Model	Test accuracy
Support Vector Machine (SVM)	_____ %
Random Forest	_____ %
Decision Tree	_____ %
Feedforward Neural Network	_____ %
Convolutional Neural Network (CNN)	_____ %
Tuned SVM	_____ %

Additional results include:

- SVM cross-validation accuracy: _____ %
- Best SVM parameters: _____
- PCA explained variance ratio: _____
- Flask API test status codes: _____

The bar chart compares the classical classifiers. The confusion matrix shows the Random Forest classification results. CNN prediction images compare actual and predicted labels. The GAN output demonstrates the generation of synthetic digit-like images.

The experiment metrics are saved in `digitlab_outputs/experiment_results.json`.

Note: Do not fill in estimated accuracy values. Use the output from your actual program execution.

11. OUTPUT FILES

The project creates an output directory named `digitlab_outputs`.

File	Description
`random_forest.joblib`	Saved Random Forest classifier
`cnn_model.keras`	Saved CNN classification model
`digit_generator.keras`	Saved GAN generator
`best_svm.joblib`	Saved tuned SVM classifier
`app.py`	Flask REST API source code
`requirements.txt`	Python dependencies for the API

`Dockerfile`	Docker image configuration
`experiment_results.json`	Recorded experiment metrics and API test results

12. ADVANTAGES

1. Combines multiple machine learning and deep learning techniques in one project.
2. Demonstrates both classical classification and neural-network-based classification.
3. Uses visualizations to compare model performance and predictions.
4. Introduces unsupervised learning through K-Means clustering.
5. Demonstrates dimensionality reduction using PCA.
6. Introduces generative modeling using a GAN.
7. Supports hyperparameter optimization through GridSearchCV.
8. Saves trained models and experimental results for future use.
9. Provides a REST API for accessing CNN predictions.
10. Generates configuration files to support future deployment.

13. LIMITATIONS

1. Classical machine learning models use only a subset of the MNIST dataset.
2. The feedforward neural network and CNN are trained for only three epochs.
3. The GAN is trained for two epochs, so generated images may not resemble clear handwritten digits.
4. K-Means clusters are not guaranteed to correspond directly to the ten digit classes.
5. The API accepts pixel arrays rather than directly processing uploaded image files.
6. The project generates a Dockerfile but does not build or deploy a Docker image.
7. The confidence score returned by the CNN is a softmax score and is not a guarantee that the prediction is correct.

14. FUTURE ENHANCEMENTS

The project can be improved in several ways:

- Train the models for more epochs and experiment with larger training subsets.
- Evaluate models using precision, recall, F1-score, and per-class metrics.
- Develop a web interface that allows users to draw or upload handwritten digits.
- Add automatic image resizing, grayscale conversion, and normalization to the API.
- Improve the GAN architecture and training procedure to generate clearer images.
- Build and deploy the Docker image in a suitable environment.
- Deploy the Flask API to a cloud platform.
- Add comprehensive input validation, logging, and automated API tests.
- Compare additional machine learning and deep learning architectures.

15. CONCLUSION

DigitLab successfully brings together multiple machine learning and deep learning techniques for handwritten digit recognition using the MNIST dataset. The project covers data preprocessing, SVM, Random Forest, Decision Tree, K-Means clustering, PCA, feedforward neural networks, CNNs, GANs, hyperparameter tuning, and REST API development.

The project provides a practical understanding of how image data can be prepared, models can be trained and evaluated, and trained models can be made accessible through an API. It also demonstrates model saving, experiment result recording, and preparation for containerized deployment.

The actual classification performance can be assessed using the accuracy values and other metrics generated during notebook execution. Overall, DigitLab provides a useful foundation for understanding the complete workflow of a machine learning project, from dataset preparation to prediction-service development.

REFERENCES

1. TensorFlow Documentation — https://www.tensorflow.org/
2. Scikit-learn Documentation — https://scikit-learn.org/
3. MNIST Database of Handwritten Digits — http://yann.lecun.com/exdb/mnist/

4. Flask Documentation — https://flask.palletsprojects.com/
5. Matplotlib Documentation — https://matplotlib.org/
6. Keras Documentation — https://keras.io/